GenCourseAI

Intro to React Hooks

Learn modern React state and lifecycle management using hooks like useState, useEffect, and custom hooks.

CURRICULUM COMPILED DYNAMICALLY BY **GENCOURSE AI** ENGINE

Table of Contents

MODULE 1

Module 1: Foundations of Hooks

1.1 The Shift to Functional Components

LESSON OBJECTIVES

- Understand the core concepts of 1.1 The Shift to Functional Components
- Learn the best practices of 1.1 The Shift to Functional Components
- Apply 1.1 The Shift to Functional Components in real-world scenarios

React Hooks were introduced in version 16.8 to allow functional components to manage state and side effects. Previously, stateful logic required complex Class Components with 'this' bindings and fragmented lifecycle methods like 'componentDidMount'. Functional components with Hooks are cleaner, shorter, and easier to reuse.

Key Takeaways:

- No more 'class' syntax or 'this' context bindings.
- Logic can be grouped by function rather than lifecycle phase.
- High-performance execution with simpler component trees.

1.2 Managing State with useState

LESSON OBJECTIVES

- Understand the core concepts of 1.2 Managing State with useState
- Learn the best practices of 1.2 Managing State with useState
- Apply 1.2 Managing State with useState in real-world scenarios

The 'useState' hook allows you to declare reactive state variables in functional components. It returns a state value and a setter function.

```
const [count, setCount] = useState(0);
```

Whenever 'setCount' is called with a new value, React schedules a re-render of the component to reflect the UI updates immediately.

MODULE 2

Module 2: Handling Side Effects

2.1 Mastering the useEffect Hook

LESSON OBJECTIVES

- Understand the core concepts of 2.1 Mastering the useEffect Hook
- Learn the best practices of 2.1 Mastering the useEffect Hook
- Apply 2.1 Mastering the useEffect Hook in real-world scenarios

Side effects include fetching data, manually updating the DOM, subscribing to events, and timers. The 'useEffect' hook runs code after the component renders.

Dependency Array Rules:

1. **No array**: Runs after every single render.
2. **Empty array []**: Runs exactly once on mount and cleanups on unmount.
3. **Variables inside [deps]**: Runs whenever any dependency changes.